

Rigid body simulator

1 Introduction

This document will describe a very simple rigid body simulator we will use to design and develop our drone control system. Too efficiently design the system we will need simulations. In this module we will design a very simple simulator which will solve Newton's equations. In doing so, we can predict under certain approximations how the drone will behave when we control it.

2 Reference system

Two reference frames are used throughout this document (the same frames as in the AHRS documentation):

- The **world frame** (n): fixed to the world. It is treated as non-inertial in general, but for the purposes of this simulator Newton's equations are written directly in it.
- The **body-fixed frame** (b): its origin is fixed somewhere on the drone (not necessarily at the center of mass) and its axes rotate with the drone.

The center of gravity (CoG) of the body is fixed in the body frame and described by the constant vector $\mathbf{c}^b$, expressed in the b frame relative to the body origin.

2.1 Orientation and the rotation matrix

The orientation of the body is represented by a unit quaternion $\mathbf{p} = [p_0, p_1, p_2, p_3]^T$ (scalar-first convention), describing the rotation of the body frame relative to the world frame. From the quaternion we can construct the rotation matrix $\mathbf{R}_{nb}$, which transforms body-frame vectors into the world frame:

$$\mathbf{v}^n = \mathbf{R}_{nb} \mathbf{v}^b, \quad \mathbf{v}^b = \mathbf{R}_{nb}^T \mathbf{v}^n \quad (1)$$

For a unit quaternion $\mathbf{p} = [p_0, p_1, p_2, p_3]^T$ the rotation matrix is given by:

$$\mathbf{R}_{nb} = \begin{bmatrix} 1 - 2(p_2^2 + p_3^2) & 2(p_1p_2 - p_0p_3) & 2(p_1p_3 + p_0p_2) \\ 2(p_1p_2 + p_0p_3) & 1 - 2(p_1^2 + p_3^2) & 2(p_2p_3 - p_0p_1) \\ 2(p_1p_3 - p_0p_2) & 2(p_2p_3 + p_0p_1) & 1 - 2(p_1^2 + p_2^2) \end{bmatrix} \quad (2)$$

2.2 Directions versus points

When transforming quantities between frames we must distinguish two kinds of vectors. Throughout this document a location is denoted by $\mathbf{s}$; the symbol $\mathbf{x}$ is reserved for the simulator state vector introduced in Section 3.

- **Free vectors** (forces, torques, velocities, angular velocities): only the direction and magnitude matter, so they transform by rotation alone, $\mathbf{v}^n = \mathbf{R}_{nb} \mathbf{v}^b$.
- **Points** (force application points, sensor positions): these denote a location and additionally require a translation. A body-frame point $\mathbf{s}^b$ is expressed relative to the *body origin*. Since the simulator state stores the world position of the center of mass, $\mathbf{r}$, the corresponding world-frame point is

$$\mathbf{s}^n = \mathbf{r} + \mathbf{R}_{nb} (\mathbf{s}^b - \mathbf{c}^b) \quad (3)$$

where the CoG offset $\mathbf{c}^b$ accounts for the fact that the body origin and the center of mass do not coincide in general. A world-frame point is simply an absolute position and needs no transformation.

Because the state tracks the center of mass while the user describes the geometry (CoG, sensor positions, force application points) relative to the body origin, the initial condition also requires this conversion: given the initial world position of the body origin $\mathbf{r}_{\text{origin}}$ and initial orientation $\mathbf{p}_0$, the initial center of mass position is

$$\mathbf{r}_0 = \mathbf{r}_{\text{origin}} + \mathbf{R}_{nb}(\mathbf{p}_0) \mathbf{c}^b \quad (4)$$

3 Rigid body dynamics

In this section, we discuss the rigid-body dynamics required to solve the equations of motion. Throughout this chapter, all vectors are expressed in the non-inertial world reference frame, denoted by n .

The objective is to determine the system state, which is defined by the state vector

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \mathbf{v} \\ \mathbf{p} \\ \boldsymbol{\omega} \end{bmatrix},$$

where

- $\mathbf{r}$ is the position of the body's center of mass in the world frame,
- $\mathbf{v}$ is the linear velocity of the center of mass,
- $\mathbf{p}$ is the orientation of the body, represented as a quaternion,
- $\boldsymbol{\omega}$ is the angular velocity of the body, expressed in the world frame.

At the start of the simulation, the state vector is initialized to a prescribed initial condition, $\mathbf{x}_0$, chosen to represent the scenario under consideration.

The evolution of the system is governed by the equations of motion, which define the time derivative of the state vector,

$$\dot{\mathbf{x}} = \begin{bmatrix} \mathbf{v} \\ \mathbf{a} \\ \dot{\mathbf{p}} \\ \boldsymbol{\alpha} \end{bmatrix}$$

- $\mathbf{v}$ is the linear velocity of the center of mass,
- $\mathbf{a}$ is the acceleration of the body
- $\dot{\mathbf{p}}$ is the orientation rate of change of the body,
- $\boldsymbol{\alpha}$ is the angular acceleration of the body, expressed in the world frame.

At each simulation step, the state derivative $\dot{\mathbf{x}}$ is computed from the current state $\mathbf{x}$ together with the applied external forces and moments. A numerical integration scheme is then used to advance the state from the current time step to the next. This procedure is repeated iteratively until the desired simulation time has been reached.

In the next sections we will discuss how to obtain each element of the state derivative vector. After that we discuss two integration schemes which can be used too obtain the next state.

3.1 External forces and torques in arbitrary frames

An external load on the body is described either as a pure torque $\boldsymbol{\tau}_i$, or as a force $\mathbf{F}_j$ together with its application point $\mathbf{s}_j$. Both the force vector and its application point may be expressed in *any* of the frames defined above; this is a convenience of the interface, not of the physics. Before the equations of motion are evaluated, everything is brought into the world frame:

- force and torque vectors are rotated, $\mathbf{F}_j^n = \mathbf{R}_{nb} \mathbf{F}_j^b$ (and analogously for $\boldsymbol{\tau}_i$),
- application points are transformed as points using Eq. (3).

The moment arm of a force about the center of mass is then

$$\mathbf{r}_j = \mathbf{s}_j^n - \mathbf{r} \quad (5)$$

i.e. the world-frame vector from the center of mass to the application point. Note that a force applied exactly at the CoG ($\mathbf{s}_j^b = \mathbf{c}^b$) has $\mathbf{r}_j = \mathbf{0}$ and therefore produces no torque — this is how, for example, gravity is applied.

It is important that this transformation into the world frame is performed with the orientation of the state *currently being evaluated*, not with a fixed orientation: a body-fixed force (such as propeller thrust) must rotate together with the body. This distinction matters for the multi-stage integration schemes of Section 5, where the derivative is evaluated at intermediate candidate states. Within a single time step, the set of applied forces itself is held constant (a zero-order hold); only its expression in the world frame changes with the evaluated orientation.

3.2 Computing linear acceleration

We start by computing the total force $\mathbf{F}$ (described in the n frame) by summing all the known external forces acting on the body:

$$\mathbf{F} = \sum_i \mathbf{F}_i \quad (6)$$

Now we can compute the acceleration the rigid body experiences:

$$\mathbf{a} = \frac{\mathbf{F}}{m}$$

In this case m is the total mass of the rigid body.

3.3 Computing angular acceleration

To determine the angular acceleration, we first compute the net torque $\boldsymbol{\tau}$ acting on the rigid body about its center of mass. This is achieved by summing all pure torques alongside the moments generated by forces applied at a distance from the center of mass:

$$\boldsymbol{\tau} = \sum_i \boldsymbol{\tau}_i + \sum_j (\mathbf{r}_j \times \mathbf{F}_j) \quad (7)$$

where $\mathbf{r}_j$ is the moment arm of Eq. (5), the position vector from the body's center of mass to the point of application of the force $\mathbf{F}_j$, and $\times$ denotes the vector cross product. All quantities are assumed to have already been expressed in the world frame as described above.

Because the equations of motion are expressed in the world reference frame n , the constant body-frame inertia tensor $\mathbf{I}_{\text{body}}$ must be transformed into the world frame at each time step. Using the rotation matrix $\mathbf{R}$ derived from the current orientation quaternion $\mathbf{p}$, the world inertia tensor $\mathbf{I}$ is obtained via the following change of basis:

$$\mathbf{I} = \mathbf{R} \mathbf{I}_{\text{body}} \mathbf{R}^T \quad (8)$$

where $\mathbf{R}^T$ is the transpose of the rotation matrix.

The angular acceleration $\boldsymbol{\alpha}$ can then be derived using Euler's equations of motion in the world frame, accounting for both the net torque and gyroscopic effects:

$$\boldsymbol{\alpha} = \mathbf{I}^{-1} (\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{I} \boldsymbol{\omega})) \quad (9)$$

where $\boldsymbol{\omega}$ is the current angular velocity in the world frame.

3.4 Computing orientation derivative

The rate of change of the orientation quaternion, $\dot{\mathbf{p}}$, is determined directly from the current angular velocity $\boldsymbol{\omega}$ and the current orientation $\mathbf{p}$. Because the angular velocity vector is expressed in the world frame n , the quaternion derivative is given by:

$$\dot{\mathbf{p}} = \frac{1}{2} \boldsymbol{\omega}_q \otimes \mathbf{p} \quad (10)$$

where $\otimes$ denotes the quaternion multiplication operator, and $\boldsymbol{\omega}_q$ is the angular velocity vector promoted to a pure quaternion with a zero scalar component:

$$\boldsymbol{\omega}_q = \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} \quad (11)$$

In practice, this quaternion multiplication can be written as a matrix-vector product to make numerical implementation straightforward:

$$\dot{\mathbf{p}} = \frac{1}{2} \begin{bmatrix} -\boldsymbol{\omega} \cdot \mathbf{p}_{1:3} \\ p_0 \boldsymbol{\omega} + \boldsymbol{\omega} \times \mathbf{p}_{1:3} \end{bmatrix} \quad (12)$$

where p_0 is the scalar component of the quaternion $\mathbf{p}$, and $\mathbf{p}_{1:3}$ represents its vector component.

Because numerical integration schemes do not naturally preserve the unit length of a quaternion, the orientation quaternion $\mathbf{p}$ must be re-normalized to unit length ($\|\mathbf{p}\| = 1$) at the end of every simulation step to prevent numerical drift from distorting the rotation.

3.5 Linear and angular velocities

By examining the state vector $\mathbf{x}$ and its time derivative $\dot{\mathbf{x}}$, it becomes apparent that determining the linear and angular velocities for the state derivative equation $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ is straightforward. Rather than requiring dynamic computation from external forces or torques, these velocity terms are already explicitly stored as components within the state vector itself.

4 Kinematics of an arbitrary body-fixed point

The state describes the motion of the center of mass, but for developing the control system we need to know what a sensor mounted somewhere on the body would experience — for instance an IMU that is not located at the CoG. Consider a point P that is fixed in the body frame, and let

$$\boldsymbol{\rho} = \mathbf{s}_P^n - \mathbf{r} \quad (13)$$

be the world-frame vector from the center of mass to that point, with $\mathbf{s}_P^n$ obtained from the point transform of Eq. (3).

4.1 Angular velocity at a point

The angular velocity is a property of the rigid body as a whole: every point of the body rotates with the same $\boldsymbol{\omega}$. The position of the query point therefore does not change the result; it only determines the frame in which the answer is expressed ($\boldsymbol{\omega}$ in the world frame, or $\mathbf{R}_{nb}^T \boldsymbol{\omega}$ in the body frame).

4.2 Acceleration at a point

For a point rigidly attached to the body, standard rigid-body kinematics gives the acceleration

$$\mathbf{a}_P = \mathbf{a} + \boldsymbol{\alpha} \times \boldsymbol{\rho} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{\rho}) \quad (14)$$

where all vectors are expressed in the world frame:

- $\mathbf{a}$ is the acceleration of the center of mass ($\mathbf{a} = \mathbf{F}/m$),
- $\boldsymbol{\alpha} \times \boldsymbol{\rho}$ is the tangential (Euler) acceleration caused by the angular acceleration,
- $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{\rho})$ is the centripetal acceleration caused by the rotation itself.

Because P is fixed in the body there is no relative velocity of the point with respect to the body, and hence no Coriolis ($2\boldsymbol{\omega} \times \mathbf{v}_{\text{rel}}$) term. The values of $\mathbf{a}$ and $\boldsymbol{\alpha}$ are obtained by evaluating the state derivative with the currently applied forces and torques.

Like the applied forces, the query point may be given in any frame, and the resulting acceleration is returned expressed in that same frame. Note that Eq. (14) is the *kinematic* (coordinate) acceleration of the point; an actual IMU measures specific force, which additionally involves subtracting gravity. That belongs to a separate sensor model built on top of this simulator.

5 Integration Schemes

In this section, we describe the two integration schemes implemented in our simulator: the Forward Euler method and the fourth-order Runge-Kutta (RK4) method.

5.1 Forward Euler

The Forward Euler integration scheme is the simplest explicit method for solving ordinary differential equations. Given the current state $\mathbf{x}_n$ and its time derivative $\dot{\mathbf{x}}_n = f(t_n, \mathbf{x}_n)$, the next state $\mathbf{x}_{n+1}$ is computed using the following equation:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t \cdot \dot{\mathbf{x}}_n \quad (15)$$

While computationally lightweight and straightforward to implement, this first-order method is highly susceptible to accumulation errors and numerical instability. It assumes that the state derivative remains constant over the entire time interval Δt , which rarely holds true in dynamic systems.

5.2 Fourth-Order Runge-Kutta (RK4)

To achieve higher accuracy and stability, the simulator also implements the fourth-order Runge-Kutta (RK4) method. Rather than relying solely on the derivative at the beginning of the time step, RK4 samples the derivative $f(t, \mathbf{x})$ at four different points across the interval to compute a weighted average slope.

The update step is defined as follows:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (16)$$

where the intermediate slopes $\mathbf{k}_i$ are evaluated sequentially:

$$\mathbf{k}_1 = f(t_n, \mathbf{x}_n) \quad (17)$$

$$\mathbf{k}_2 = f\left(t_n + \frac{\Delta t}{2}, \mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1\right) \quad (18)$$

$$\mathbf{k}_3 = f\left(t_n + \frac{\Delta t}{2}, \mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2\right) \quad (19)$$

$$\mathbf{k}_4 = f(t_n + \Delta t, \mathbf{x}_n + \Delta t\mathbf{k}_3) \quad (20)$$

RK4 systematically samples the slope four separate times across the time step. By doing this clever averaging the simulation ends up being more accurate and stable.

Naturally, this increased accuracy comes at a higher computational cost. While Forward Euler requires only a single derivative evaluation per time step, RK4 requires four, making each step four times more expensive to compute.

6 Numerical implementation notes

A few consequences of the mathematics above shape how the simulator is structured.

6.1 The derivative as a pure function of the state

Both integration schemes are written in terms of a derivative function $f(t, \mathbf{x})$ that can be evaluated at *any* candidate state, not only at the currently stored one. This is essential for RK4: the stages $\mathbf{k}_2$ – $\mathbf{k}_4$ evaluate the dynamics at intermediate states such as $\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1$ that are never adopted as the actual state of the system. The rigid body therefore computes its state derivative from the state vector it is handed — in particular, body-frame forces are rotated into the world frame using the *candidate* orientation quaternion contained in that state — and its stored state is only overwritten once per step with the final result. This separation also keeps the integrators generic: any system that can express itself as a state vector with a derivative function can be registered with a solver and advanced alongside the rigid body.

6.2 Quaternion normalization

As noted in the discussion of the orientation derivative, numerical integration does not preserve the unit length of the quaternion, so $\mathbf{p}$ is re-normalized when the final state of a step is stored. In addition, the intermediate RK4 candidate states contain quaternions that are slightly non-unit (they are sums of a unit quaternion and scaled derivative terms). Whenever a rotation matrix is constructed during a derivative evaluation, the quaternion is therefore normalized first, so that $\mathbf{R}_{nb}$ is always a proper rotation matrix even at intermediate stages.

6.3 Force application over a step

The applied forces and torques are treated as constant over a time step (a zero-order hold): all derivative evaluations within one step see the same set of applied loads. In a typical control loop the forces are updated once per step, before the integrator advances the state. Body-frame loads are nevertheless re-expressed in the world frame at every derivative evaluation, using the orientation of the state being evaluated, as discussed above.